where the keys correspond to the names in the query:

```
name     = params[:name]
pay_type = params[:pay_type]
pos = Order.where("name = :name and pay_type = :pay_type",
                  pay_type: pay_type, name: name)
```

We can take this a step further. Because `params` is effectively a hash, we can simply pass it all to the condition. If we have a form that can be used to enter search criteria, we can use the hash of values returned from that form directly:

```
pos = Order.where("name = :name and pay_type = :pay_type",
                  params[:order])
```

We can take this even further. If we pass just a hash as the condition, Rails generates a `where` clause using the hash keys as column names and the hash values as the values to match. Thus, we could have written the previous code even more succinctly:

```
pos = Order.where(params[:order])
```

Be careful with this latter form of condition: it takes *all* the key-value pairs in the hash you pass in when constructing the condition. An alternative would be to specify which parameters to use explicitly:

```
pos = Order.where(name: params[:name],
                  pay_type: params[:pay_type])
```

Regardless of which form of placeholder you use, Active Record takes great care to quote and escape the values being substituted into the SQL. Use these forms of dynamic SQL, and Active Record will keep you safe from injection attacks.

Using Like Clauses

We might be tempted to use parameterized `like` clauses in conditions:

```
# Doesn't work
User.where("name like '?%'", params[:name])
```

Rails doesn’t parse the SQL inside a condition and so doesn’t know that the name is being substituted into a string. As a result, it will go ahead and add extra